



University of Monastir

Faculty of Sciences of Monastir

UrgentFlow
**Design and Implementation of an Intelligent Web
Platform for Medical Appointment and Queue
Management**

Final Year Project Report

Prepared by:

Mariem Sebai

Supervisor:

Mrs. Dorra Kechrid

Academic Year: 2025 – 2026

Table of contents

I. Table of contents.....	2
II. Table of figures.....	4
III. Table of tables.....	5
1.General Introduction	6
1.1 Project Objective	6
1.2 Report Structure	7
2. General project framework	8
2.1 Context and Problem Statement	8
2.2 State of the Art	8
2.3 Proposed Solution	9
2.4 Development Methodology	9
3. Requirements analysis and specification	10
3.1 Actor Identification	10
3.2 Functional Requirements :.....	12
3.3 Spécification des besoins fonctionnels.....	11
3.4 Non-Functional Requirements	13
4. System design	14
4.1 General Architecture	14
4.2 Development Environment	15
4.3 Tools Used.....	15
5. Implementation of sprints	15
5.1 Sprint 1: Authentication & Account Management	15
5.1.1 Sprint 1 Objective	15
5.1.2 Sprint 1 Backlog	16
5.1.3 Sprint 1 Analysis	16
5.1.4 Sprint 1 design	17
5.1.5 Completion of Sprint 1.....	20
5.2 Sprint 2: Hospital Integration & Filtering	21

5.2.1 Sprint 2 Objective	21
5.2.2 Sprint 2 Backlog.....	21
5.2.3 Sprint 2 Analysis	22
5.2.4 Sprint 2 Design	23
5.2.5 Completion of Sprint 2	26
5.3 Sprint 3: Appointment Booking & Queue Generation	27
5.3.1 Sprint 3 Objective.....	27
5.3.2 Sprint 3 Backlog.....	27
5.3.3 Sprint 3 Analysis.....	28
5.3.4 Sprint 3 Design	29
5.3.5 Completion of Sprint 3.....	32
5.4 Sprint 4: Notifications & Emergency Management	32
5.4.1 Sprint 4 Objective	32
5.4.2 Sprint 4 Backlog	33
5.4.3 Sprint 4 Analysis	33
5.4.4 Sprint 4 Design.....	35
5.4.5 Completion of Sprint 4.....	37
5.5 Sprint 5: Administrative & Analytics Dashboard	38
5.5.1 Sprint 5 Objective	38
5.5.2 Sprint 5 backlog.....	38
5.5.3 Sprint 5 Analysis	39
5.5.4 Sprint 5 Design	40
5.5.5 Sprint 5 Completion	42
5.6 Conclusion.....	43

6. General Conclusion and Future Perspectives **44**

Table of figures

Fig 1: Global Use Case Diagram.....	11
Fig 2: Sprint 1 use case diagram	16
Fig 3: Sprint 1 class diagram.....	17
Fig 4: Registration Sequence Diagram	18
Fig 5:Login Sequence Diagram	19
Fig 6:Logout Sequence Diagram	19
Fig 7: Registration interface	20
Fig 8:Login interface.....	20
Fig 9:Home page interface.....	21
Fig 10:Sprint 2 use case diagram.....	22
Fig 11:Sprint 2 class diagram.....	23
Fig 12:Search Establishments Sequence Diagram	25
Fig 13:Filter Establishments Sequence Diagram	25
Fig 14: Open Navigation Sequence Diagram	26
Fig 15:Search and filter interface.....	26
Fig 16:Sprint 3 use case diagram.....	28
Fig 17:Sprint 3 class diagram	29
Fig 18:Appointment Booking Sequence Diagram.....	31
Fig 19:View Appointments Sequence Diagram.....	31
Fig 20:Booking form interface	32
Fig 21:Sprint 4 use case diagram.....	33
Fig 22:Sprint 4 class diagram.....	35
Fig 23: Notification Flow Sequence Diagram.....	36
Fig 24:Emergency Request Flow Sequence Diagram.....	36
Fig 25:Emergency button interface.....	37
Fig 26: Emergency dashboard for administrators interface.....	37
Fig 27:Sprint 5 use case diagram.....	39
Fig 28:Sprint 5 class diagram.....	40
Fig 29: Appointment Management Sequence Diagram	41
Fig 30: Feedback Analysis & Recommendations Sequence Diagram.....	42
Fig 31: Feedback visualization interface	42
Fig 32: Admin dashboard interface	43

Table of tables

Tab 1: Sprints Table.....	10
Tab 2: Table of actors	10
Tab 3: Sprint 1 backlog table.....	16
Tab 4: Sprint 2 backlog table.....	21
Tab 5:Sprint 3 backlog table.....	27
Tab 6:Sprint 4 backlog table	33
Tab 7: Sprint 5 backlog table.....	38

1. General Introduction

In recent years, digital technologies have significantly transformed multiple sectors, particularly the healthcare domain. The integration of digital solutions into medical services contributes to improving operational efficiency, reducing waiting times, and enhancing patient experience.

However, in Tunisia, the management of medical appointments remains largely traditional. Patients are often required to physically visit healthcare facilities early in the morning and wait for long periods without any visibility regarding their queue position or estimated waiting time. This situation leads to time loss, overcrowding in healthcare establishments, and poor organization of medical services.

In this context, our end-of-study project, **UrgentFlow**, was developed as an intelligent web-based platform designed to digitize and optimize the management of medical appointments and patient flow. The system enables users to book appointments remotely, receive a queue number, and obtain an estimated waiting time, all without the need to be physically present at the healthcare facility.

Furthermore, the platform integrates additional advanced features such as:

- Emergency request handling
- Real-time notifications
- Feedback analysis and recommendation system for system improvement

This project was carried out as part of our academic training at the Faculty of Sciences of Monastir and reflects our commitment to developing an innovative solution aimed at improving access to healthcare services and optimizing hospital workflows.

1.1 Project Objective

The main objective of this project is to design and implement a modern and user-friendly web platform that facilitates online medical appointment booking and significantly reduces waiting times in healthcare facilities.

The application provides the following functionalities:

For Guests

- Browse the landing page and platform information
- View healthcare establishments (hospitals, clinics, pharmacies)
- Search and filter establishments by city and type
- Locate nearby hospitals based on their current position
- Send emergency requests without authentication
- Access Google Maps navigation to selected pharmacies or hospitals

For Patients

- Create an account and log in securely
- Book appointments within allowed time slots (7:00 AM – 12:00 PM)
- Receive a queue number and estimated waiting time
- Track ongoing and upcoming appointments
- Receive real-time notifications before their turn
- Manage personal profile and settings
- Access appointment history
- Provide feedback about their experience

For Administrators

- Manage appointments per medical service
- Update appointment status (waiting, in progress, done, cancelled)
- Monitor patient queues in real-time
- Access archived appointments
- Emergency administrators can additionally:
 - Manage incoming emergency requests
 - Prioritize and process urgent cases

For Super Administrators

- Access all patient feedback
- Analyze recurring issues and trends
- View rating distribution and weekly signals
- Receive automated recommendations generated by the system

In addition, the system includes:

- Integration with **OpenStreetMap API** for real-time hospital data
- Supplementary data from a local file (**places.js**)
- Integration with **Google Maps** for navigation
- A scalable backend architecture with database management via **phpMyAdmin (MySQL)**

1.2 Report Structure

To achieve these objectives, this report is structured as follows:

- **General Project Framework:** Presentation of the context, problem statement, analysis of existing solutions, and proposed solution
- **Requirements Analysis and Specification:** Identification of functional and non-functional requirements, along with UML diagrams (use case, sequence, and class diagrams)

- **System Design:** Description of system architecture, static and dynamic modeling, technology choices, and user interface design
- **Implementation and Validation:** Presentation of development steps, technologies used, implemented interfaces, and testing results
- **Conclusion and Future Work:** Summary of the project and suggestions for further improvements

2. General project framework

2.1 Context and Problem Statement

In Tunisia, medical appointment management is still predominantly handled using traditional methods. Patients are required to physically visit healthcare facilities early in the day and often wait for several hours without any visibility regarding their queue position or estimated waiting time.

This situation leads to several challenges:

- **High waiting times:** Patients spend excessive time waiting before receiving medical attention
- **Lack of organization:** Absence of digital systems results in inefficient queue management
- **Unnecessary travel:** Patients must travel early even for simple consultations
- **No visibility:** Patients have no way to track their position in the queue
- **Poor emergency handling:** Urgent cases are not efficiently prioritized

These issues directly impact the quality of healthcare services and patient satisfaction.

Our project aims to address these limitations by providing a **digital platform that automates appointment booking and queue management**, while introducing intelligent features such as notifications and emergency handling.

2.2 State of the Art

Several international solutions exist for medical appointment management:

Doctolib (France)

- Strengths: Easy online booking, notification system
- Weaknesses: No real-time queue management

Zocdoc (USA)

- Strengths: Large database of healthcare providers
- Weaknesses: No dynamic queue tracking

Practo (India)

- Strengths: Advanced search and booking
- Weaknesses: Limited adaptation to public hospital systems

Although these platforms are effective, they do not address the specific challenges of the Tunisian healthcare system, particularly **on-site queue management and emergency prioritization**.

2.3 Proposed Solution

The **UrgentFlow** platform provides a comprehensive solution tailored to the Tunisian context:

1. Centralized Healthcare Data

- Dynamic retrieval of hospitals via OpenStreetMap API
- Additional local data from *places.js*

2. Intelligent Appointment Booking

- Online booking system
- Automatic queue number generation
- Estimated waiting time calculation

3. Queue Management System

- Real-time tracking of patient position
- Notifications before consultation

4. Emergency Management

- Emergency request submission (even by guests)
- Routing to nearest hospitals
- Priority handling by emergency administrators

5. Administrative Dashboard

- Service-based dashboards
- Appointment and queue management
- Emergency request processing

6. Feedback Intelligence System

- Collection of user feedback
- Automatic analysis of repeated issues
- Weekly insights and recommendations for system improvement

7. External Integration

- OpenStreetMap API for healthcare data
- Google Maps for navigation

2.4 Development Methodology

The project was developed using an **Agile methodology based on SCRUM**, allowing iterative development and flexibility.

Sprint	Description	Durée
Sprint 0	Project setup (React + Node.js)	2 weeks
Sprint 1	Authentication and account management	1 week
Sprint 2	Hospital integration and filtering	2 weeks
Sprint 3	Appointment booking and queue generation	2 weeks
Sprint 4	Notifications and emergency system	2 weeks
Sprint 5	Administrative and analytics dashboard	4 weeks
Sprint 6	Testing and optimization	3 weeks

[Tab 1: Sprints Table](#)

3. Requirements analysis and specification

3.1 Actor Identification

An actor represents any external entity that interacts with the **UrgentFlow** system.

Actor	Description
Guest	Unauthenticated user who can browse hospitals, search/filter, view nearby locations, send emergency requests, and access Google Maps navigation
Patient	Registered user who can book appointments, manage profile, receive notifications, track queue, and give feedback
Administrator	Manages appointments, queues, and service operations within a hospital
Emergency Administrator	Specialized administrator responsible for handling emergency requests
Super Administrator	Monitors system feedback, analytics, and receives AI-based recommendations

[Tab 2: Table of actors](#)

3.2 Diagramme de cas d'utilisation global :

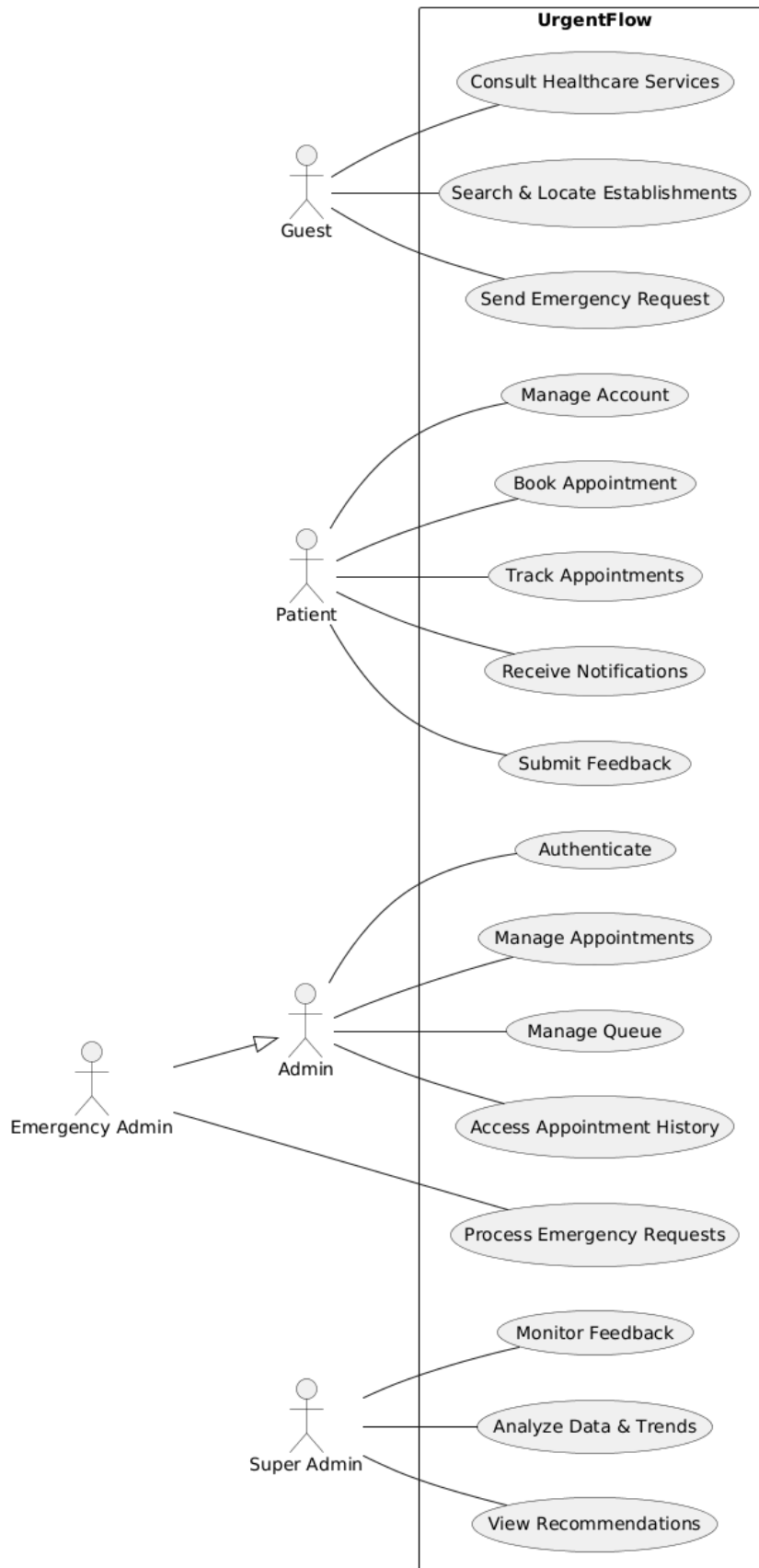


Fig 1: Global Use Case Diagram

3.3 Functional Requirements

Module 1: Authentication & Account Management

- User registration (Patient only)
- Secure login/logout (Patient & Admin)
- Redirect after registration → login → dashboard
- Profile management (update info, image)
- Session management

Module 2: Healthcare Establishments Management

- Display hospitals, clinics, pharmacies
- Fetch data via OpenStreetMap API
- Load additional data from places.js
- Search by name
- Filter by city/type
- Find nearby hospitals
- Open location in Google Maps

Module 3: Appointment Booking System

- Book appointment between 7 AM and 12 PM only
- Select hospital and service
- Generate queue number automatically
- Estimate waiting time
- Redirect to confirmation page
- View upcoming and ongoing appointments

Module 4: Queue & Notification System

- Real-time notifications
- Notify before appointment (custom delay)
- Display queue position
- Track appointment status:
 - Upcoming
 - Ongoing
 - Done
 - Cancelled

Module 5: Emergency Management

- Guest or patient can send emergency request
- System routes request to nearest hospital
- Emergency admin handles requests

Module 6: Administrative Management

- Dashboard per service
- View and manage appointments
- Update status:
 - Done
 - Ongoing
 - Cancelled (only if patient absent)
- View queue
- Access appointment archive
- Emergency dashboard (special admin)

Module 7: Feedback & Analytics System

- Patients submit feedback
- Super admin accesses all feedback
- System analyzes repeated feedback
- Displays:
 - Rating breakdown
 - Smart recommendations

3.4 Non-Functional Requirements

Performance

- Fast API response time
- Efficient loading of hospital data
- Handle concurrent users

Security

- Authentication system
- Protected backend routes
- Password hashing (bcrypt)
- Input validation

Availability

- System accessible 24/7
- Error handling mechanisms

Compatibility

- Responsive design (mobile & desktop)
- Cross-browser support

User Experience

- Simple and intuitive UI
- Smooth navigation
- Fast access to key features

Maintainability

- Modular architecture (React + Node.js)
- Clean and reusable code
- Separation of frontend/backend

4. System design

4.1 General Architecture

The **UrgentFlow** system follows a **client-server** architecture composed of three main layers:

Frontend (Client): Responsible for user interaction and interface rendering.

Backend (Server): Handles business logic and processes user requests.

Database: Stores persistent data such as users, appointments, and feedback.

The frontend communicates with the backend through REST APIs, while the backend interacts with the database to retrieve and store information.

4.2 Development Environment

The development of the project was carried out with the following tools:

Frontend

- React.js for building user interfaces
- Material UI for UI components
- JavaScript, HTML, CSS

Backend

- Node.js runtime environment
- Express.js framework for API development

Database

- MySQL for data storage
- phpMyAdmin for database administration

4.3 Tools Used

- **VS Code** → Code editor
- **Git & GitHub** → Version control
- **Postman** → API testing
- **PlantUML / StarUML** → UML diagrams
- **Google Maps API** → Navigation
- **OpenStreetMap API** → Hospital data

5. Implementation of sprints

5.1. Sprint 1: Authentication & Account Management

5.1.1 Sprint 1 Objective

The objective of this sprint is to implement a secure authentication system and enable full user account management, including registration, login, session handling, and profile management.

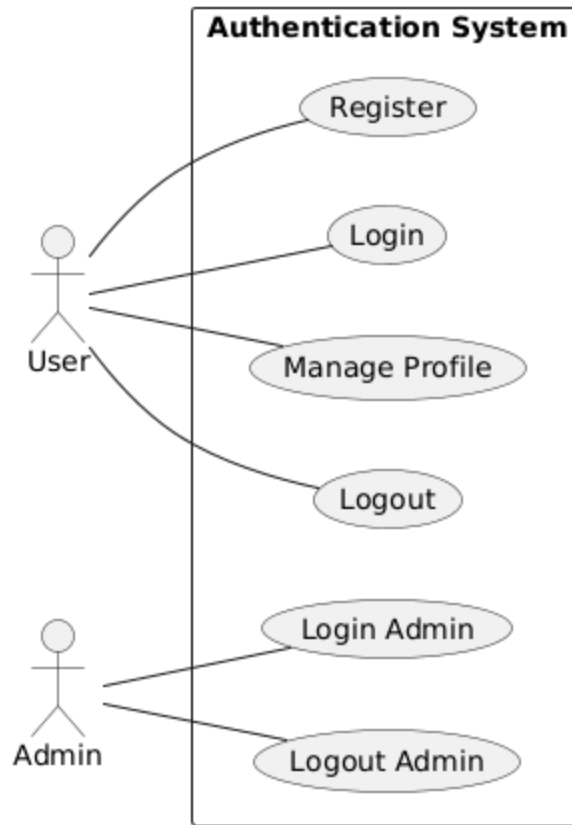
5.1.2 Sprint 1 Backlog

Id	Feature	User Story	Priority	Complexity
1	Registration	As a user, I want to create an account to access the platform	High	Medium
2	Login	As a user, I want to log into my account securely	High	Low
3	Session Management	As a user, I want to stay connected after login	High	Medium
4	Profile Management	As a user, I want to update my personal information	Medium	Medium
5	Logout	As a user, I want to securely log out	High	Low

[Tab 3: Sprint 1 backlog table](#)

5.1.3 Sprint 1 Analysis

Use case diagram:



[Fig 2: Sprint 1 use case diagram](#)

Description of scenarios:

Scenario: Register

1. The user accesses the registration page
2. He enters his personal information
3. He validates the form
4. The system checks the data
5. The account is created in the database
6. The user is redirected to the login page

Scenario: Log in

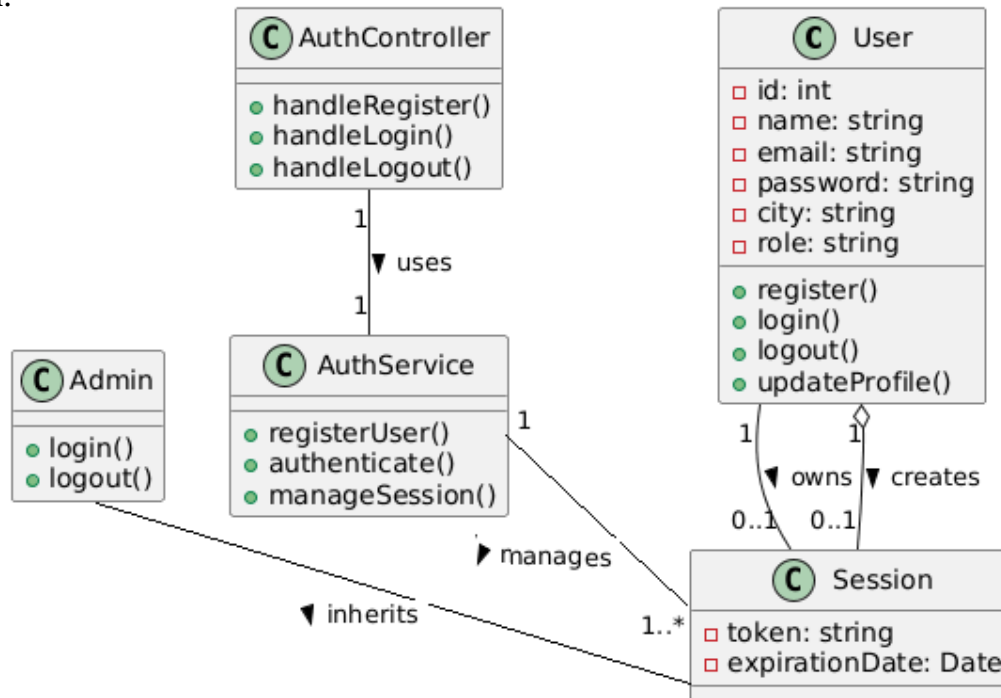
1. The user accesses the login page
2. He enters email and password
3. The system checks the information
4. If correct → access to the user dashboard
5. Otherwise → error message

Scenario: Password Recovery

1. The user enters their email
2. The system verifies the existence of the account
3. A reset code is sent
4. The user can reset their password

5.1.4 Sprint 1 design

Class Diagram:



[Fig 3: Sprint 1 class diagram](#)

Class diagram description:

- **User :**
This class represents a system user in the UrgentFlow platform. A user is characterized by attributes such as id, full name, email, password, city, and role (patient or administrator). The user can register, log in, log out, and update their profile information.
- **Admin :**
This class represents an administrator responsible for managing system operations. It is a specialized type of user (inheritance from User). An administrator can authenticate and manage system-related actions depending on their role.
- **AuthService:**
This class encapsulates the business logic related to authentication and account management. It handles user registration, credential verification, session management, and overall authentication processes.

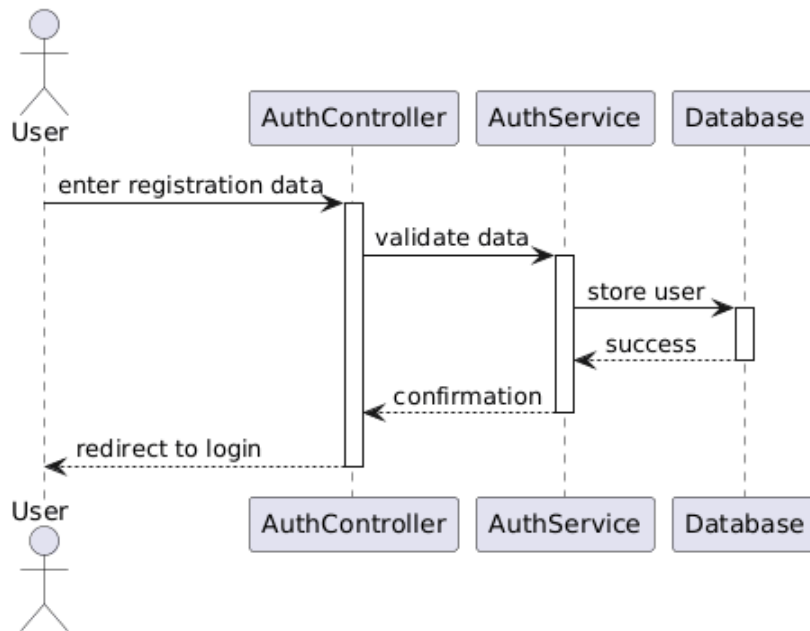
- **AuthController :**

This class acts as an interface between the frontend and the backend services. It receives user actions (registration, login, logout, profile updates) and delegates them to the AuthService for processing.

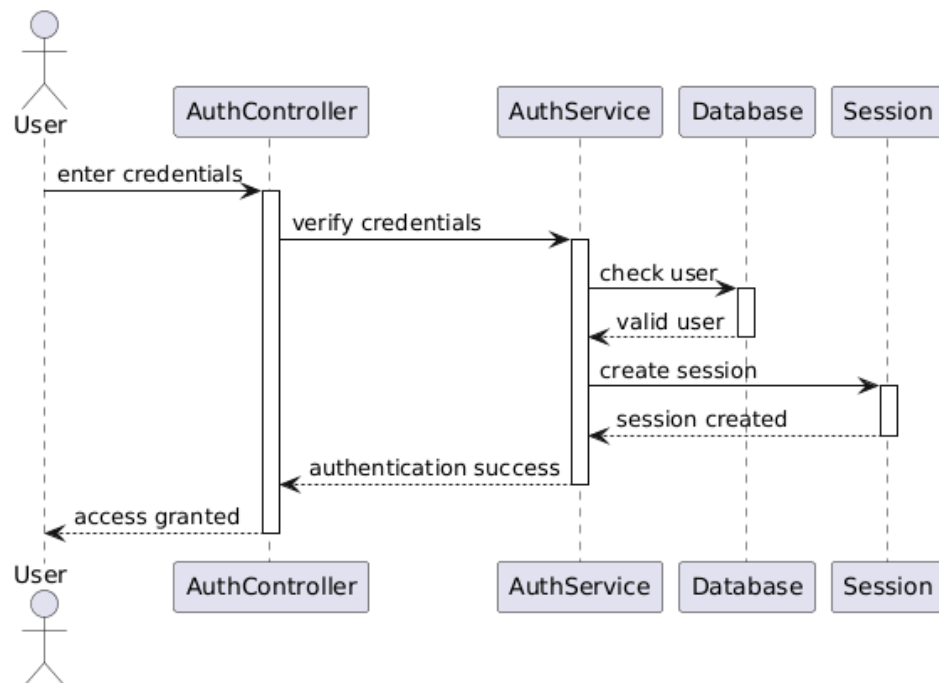
- **Session :**

This class represents an authenticated user session. It contains a unique access token and an expiration date to ensure secure access to the system. Each session is associated with a single user.

Sprint 1 sequence diagrams:



[Fig 4: Registration Sequence Diagram](#)



[Fig 5: Login Sequence Diagram](#)

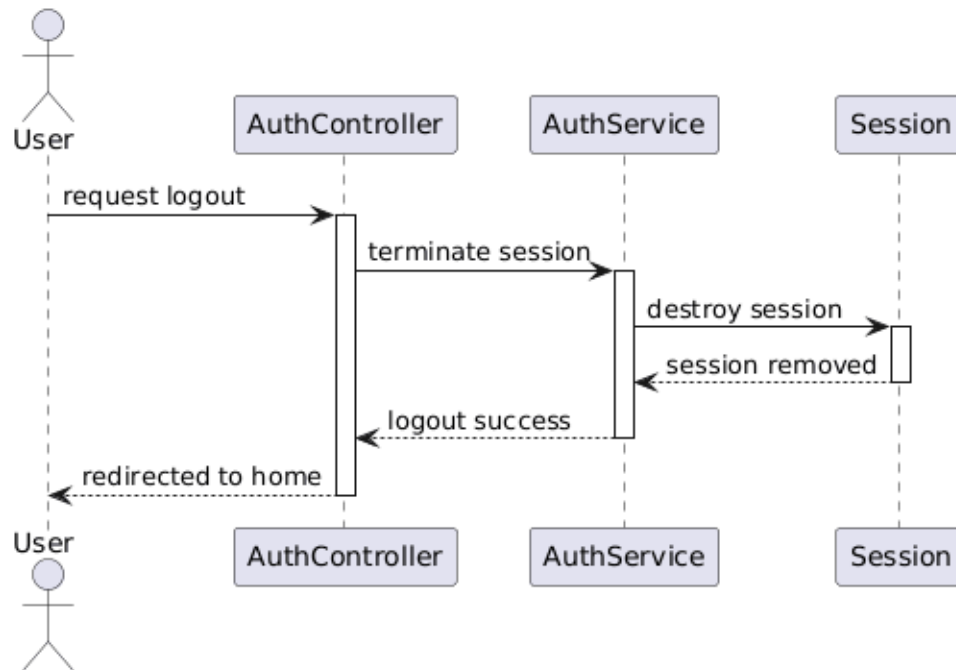
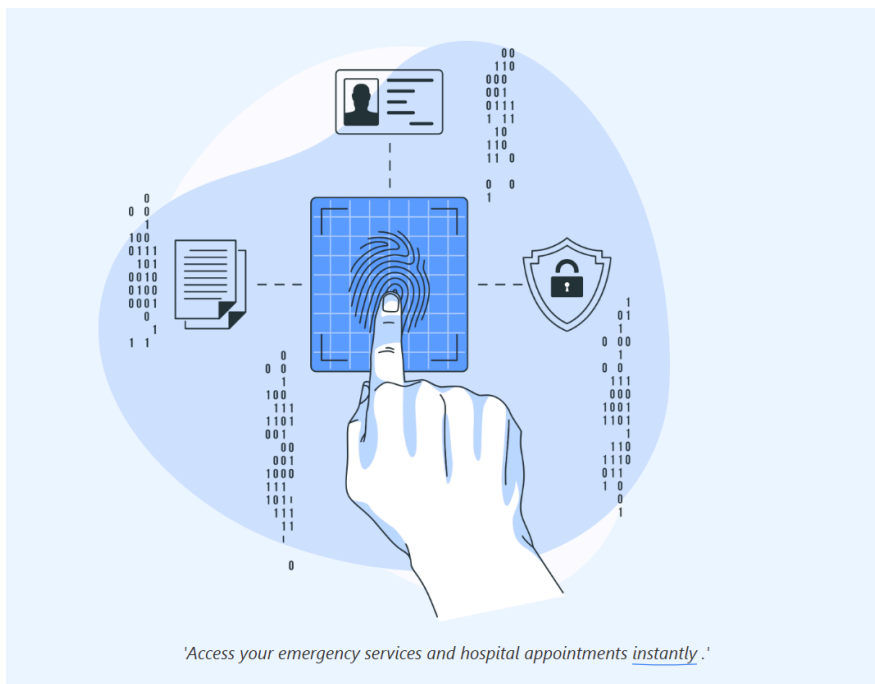


Fig 6: Logout Sequence Diagram

5.1.5 Completion of Sprint 1

Interfaces :



Welcome back
Sign in to UrgentFlow

Enter Details

☐ Remember me

[Forgot Password?](#)

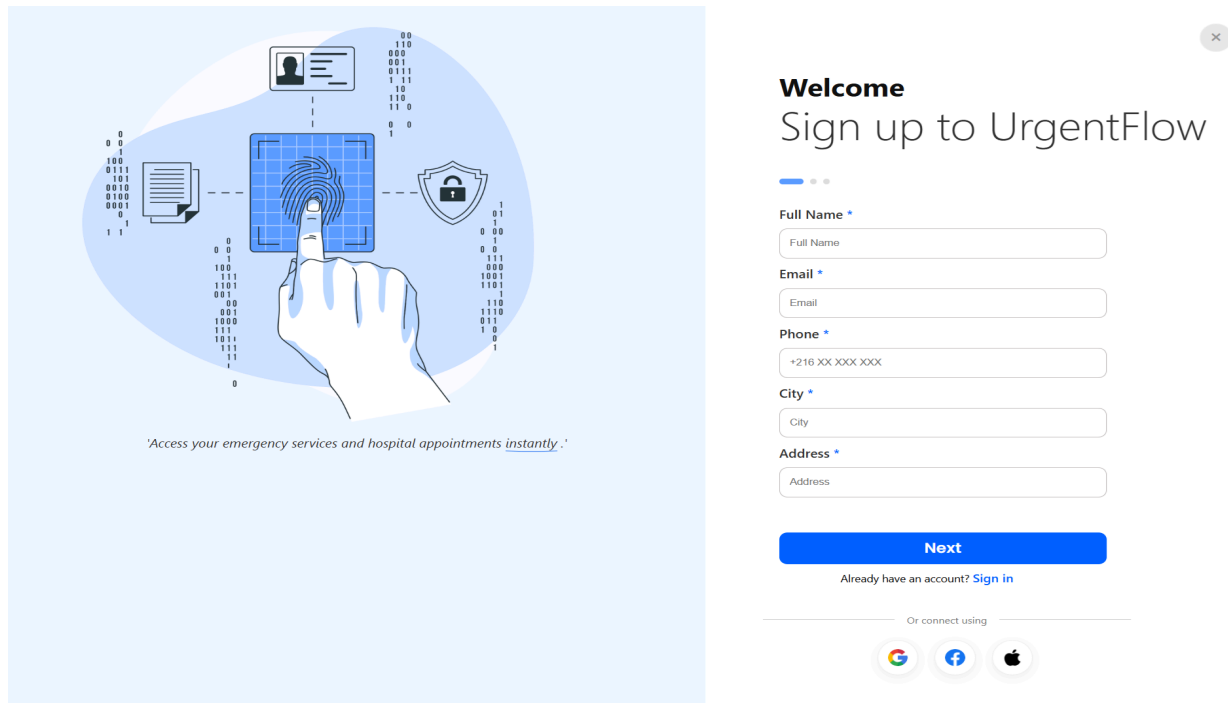
Login

Don't have an account? [Create an account](#)

Or connect using



Fig 7: Registration interface



Access your emergency services and hospital appointments instantly.

Welcome

Sign up to UrgentFlow

Full Name *

Email *

Phone *

City *

Address *

Next

Already have an account? [Sign in](#)

Or connect using

Google Facebook Apple

Fig 8: Login interface

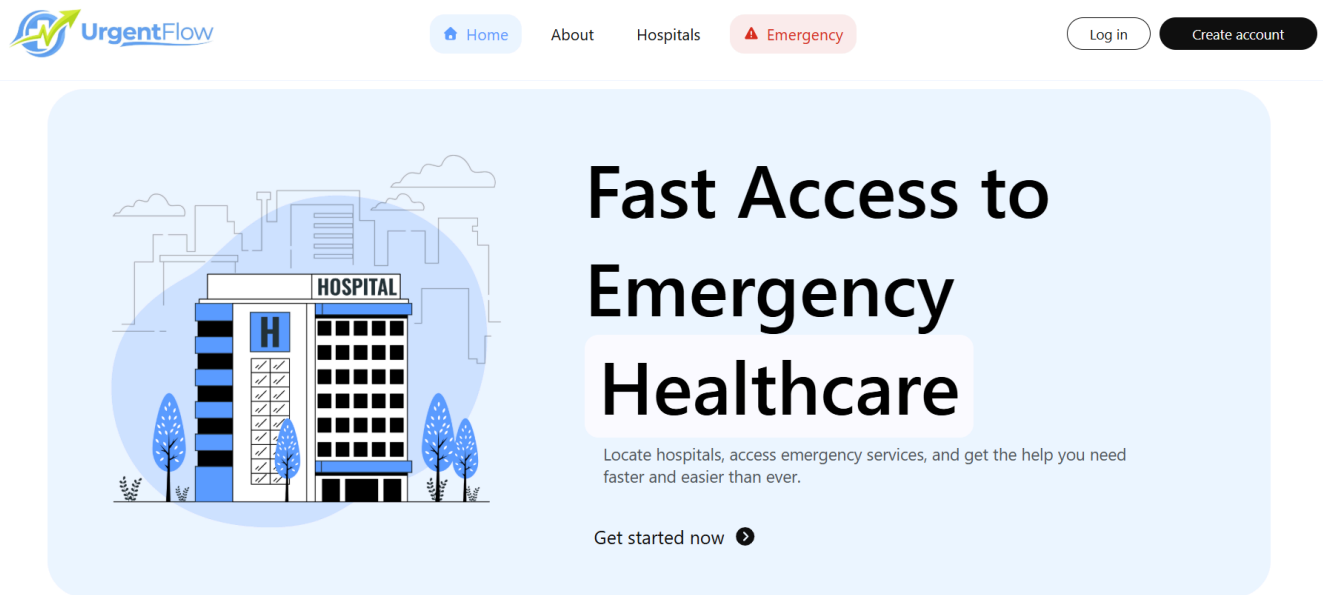


Fig 9: Home page interface

5.2 Sprint 2: Hospital Integration & Filtering

5.2.1 Sprint 2 Objective

The objective of this sprint is to implement a module that allows users to browse, search, and filter healthcare establishments (hospitals, clinics, pharmacies) in Tunisia using data retrieved from external sources such as **OpenStreetMap**.

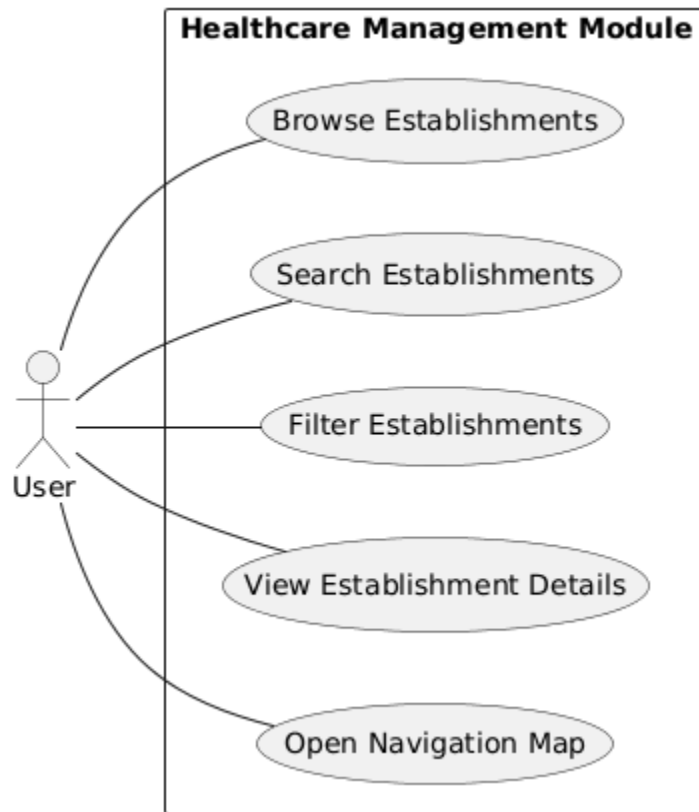
5.2.2 Sprint 2 Backlog

Id	Feature	User story	Priority	Complexity
1	View establishments	As a user, I want to view a list of healthcare facilities	High	Medium
2	Search	As a user, I want to search by name or city	High	Medium
3	Filter by type	As a user, I want to filter by hospital, clinic, or pharmacy	High	Medium
4	Filter by city	As a user, I want to filter by location	High	High
5	Combine filters	As a user, I want to apply multiple filters simultaneously	Medium	High
6	Navigation	As a user, I want to open directions in Google Maps	High	Medium

[Tab 4: Sprint 2 backlog table](#)

5.2.3 Sprint 2 Analysis

Use case diagram:



[Fig 10: Sprint 2 use case diagram](#)

Description of scenarios:

Scenario: Browse Establishments

The user accesses the establishments page.

The system displays a list of healthcare facilities retrieved from external data sources.

The user can scroll through the list and view basic information such as name, type, and city.

Scenario: Search Establishments

The user enters a keyword (name or city) in the search bar.

The system processes the request and retrieves matching establishments.

The filtered results are displayed dynamically to the user.

Scenario: Filter Establishments

The user selects one or more filtering criteria (type, city).

The system applies the selected filters to the dataset.

The list of establishments is updated in real time according to the selected criteria.

Scenario: Combine Filters

The user applies multiple filters simultaneously (e.g., city + type).

The system processes all criteria together.

Only establishments matching all selected filters are displayed.

Scenario: View Establishment Details

The user selects a specific establishment from the list.

The system displays detailed information including location and type.

The user can decide to navigate to the selected establishment.

Scenario: Open Navigation (Google Maps)

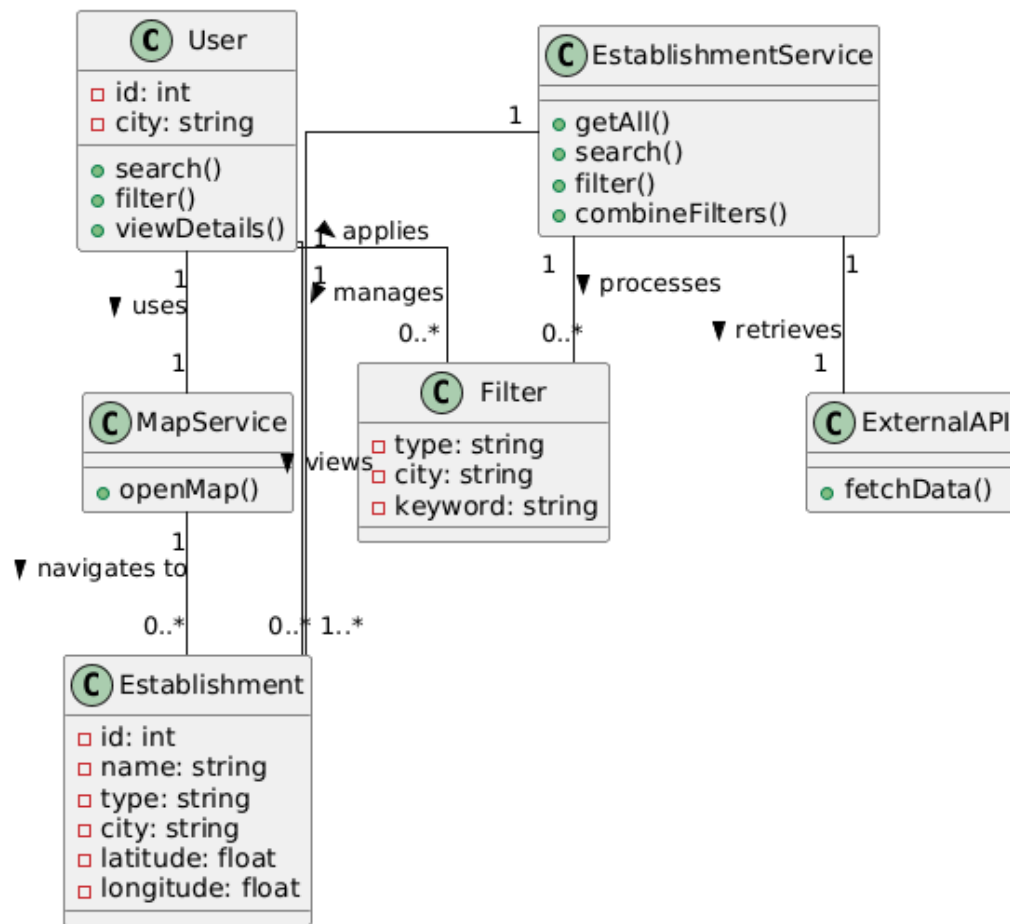
The user clicks on the navigation button.

The system generates a map link using the establishment's coordinates.

The user is redirected to Google Maps to view directions.

5.2.4 Sprint 2 Design

Class Diagram:



[Fig 11: Sprint 2 class diagram](#)

Class diagram description:

User

This class represents a platform user interacting with the system.

The user can search, filter, and browse healthcare establishments, as well as access navigation services.

Establishment

This class represents a healthcare facility (hospital, clinic, or pharmacy).

It is characterized by attributes such as id, name, type, city, and geographical coordinates (latitude and longitude).

It serves as the main entity displayed and manipulated in this module.

EstablishmentService

This class contains the business logic related to healthcare establishments.

It is responsible for retrieving data, performing search operations, applying filters, and combining multiple filtering criteria.

It acts as the core component connecting the system to external data sources.

Filter

This class represents filtering criteria applied by the user.

It includes attributes such as type, city, and keyword.

It allows flexible and dynamic filtering of establishments based on user preferences.

MapService

This class manages navigation functionality.

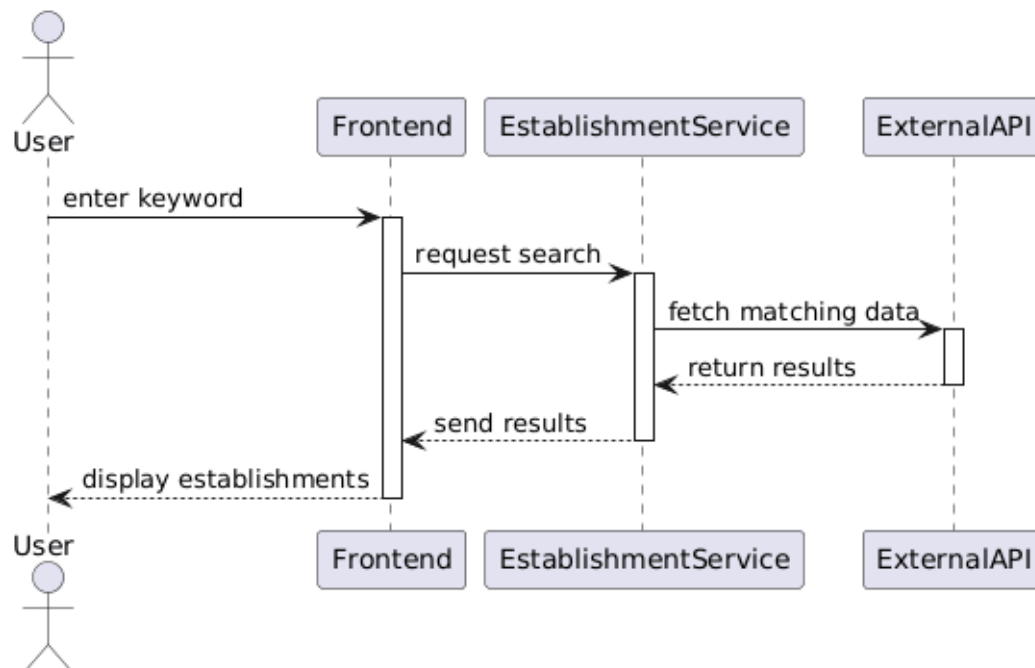
It generates map links using the geographical coordinates of establishments and allows users to access directions via Google Maps.

ExternalAPI

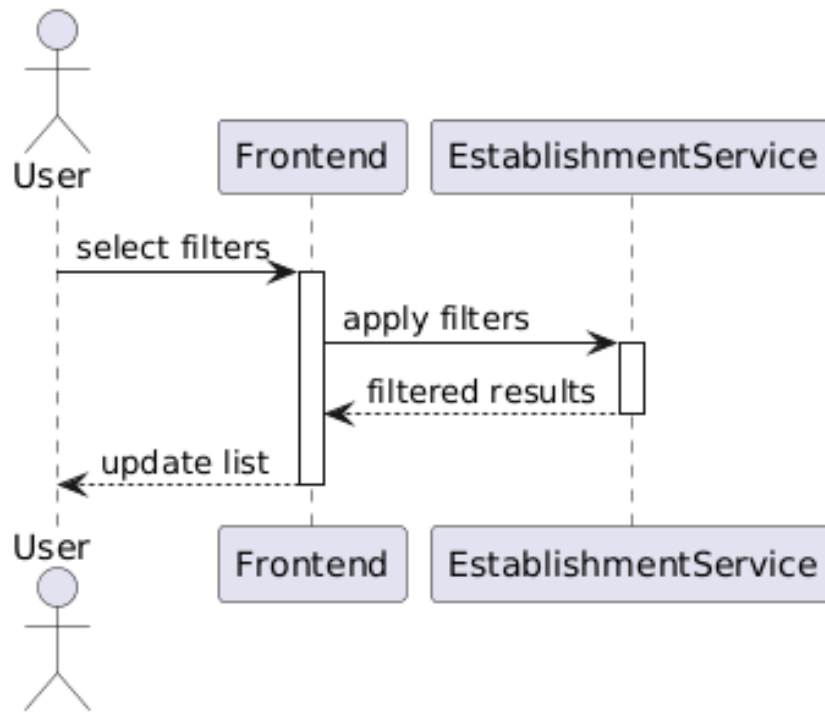
This class represents the external data source used to retrieve healthcare establishments (e.g., OpenStreetMap).

It provides raw data that is processed and structured by the system before being presented to the user.

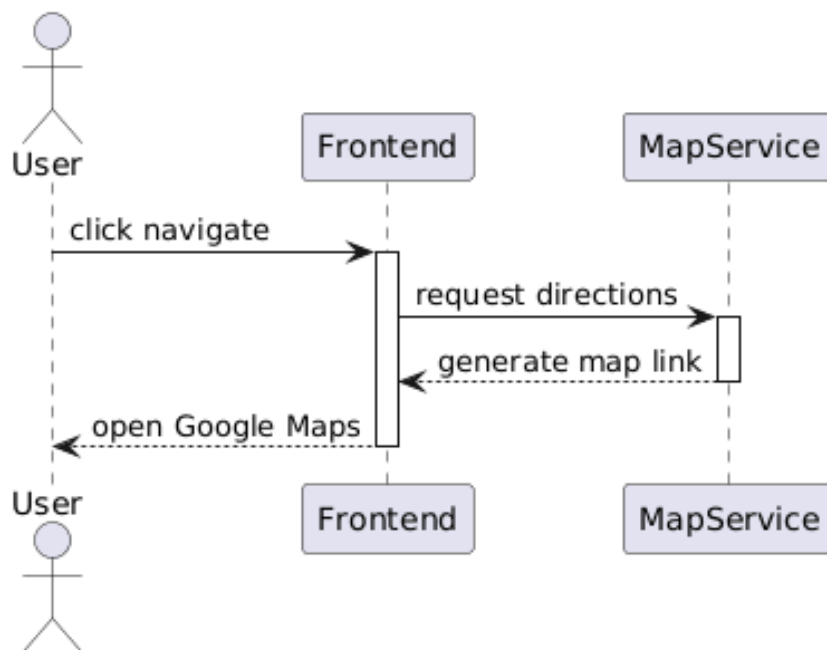
Sprint 2 sequence diagrams:



[Fig 12: Search Establishments Sequence Diagram](#)



[Fig 13: Filter Establishments Sequence Diagram](#)



[Fig 14: Open Navigation Sequence Diagram](#)

5.2.5 Completion of Sprint 2

Interfaces :

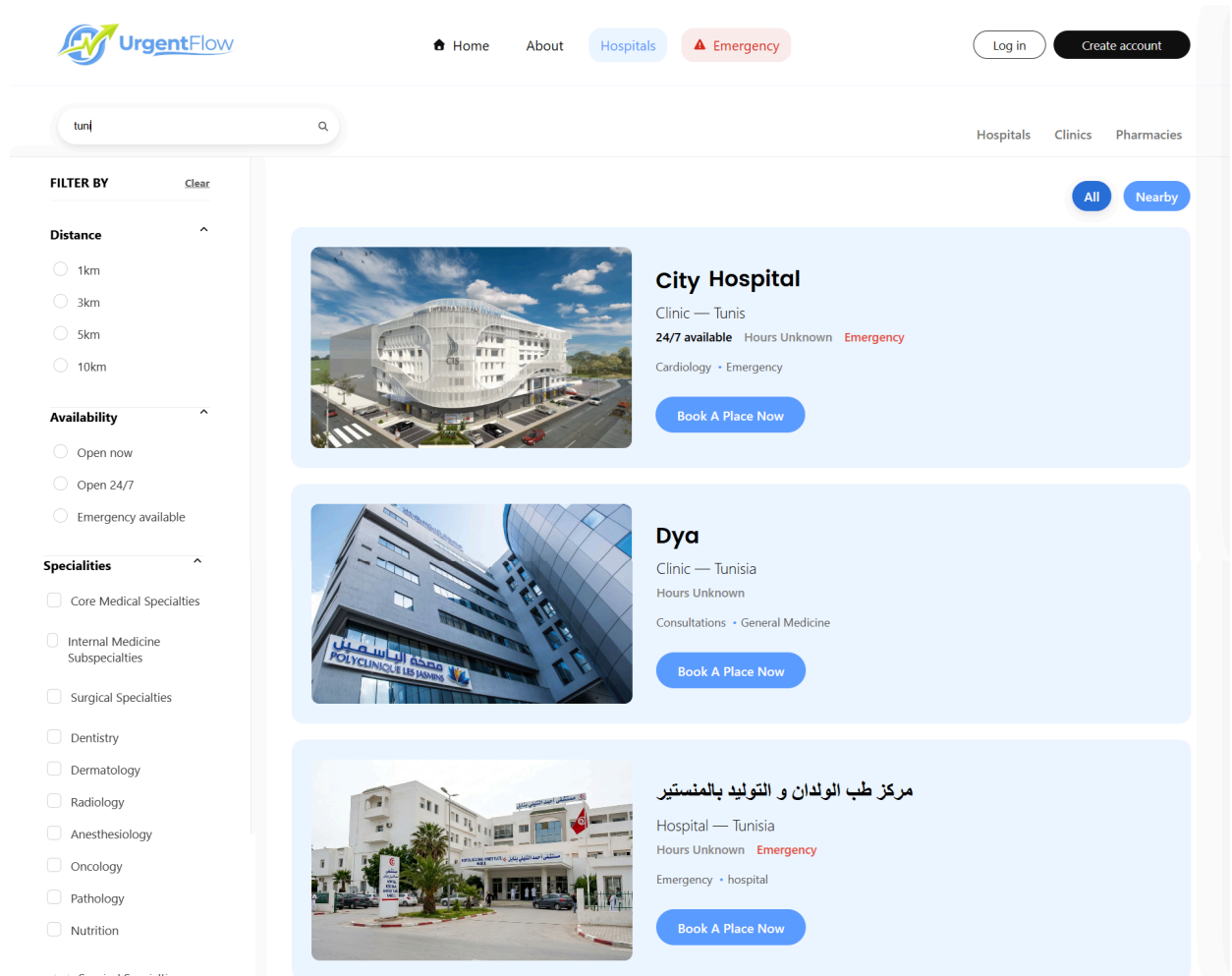


Fig 15: Search and filter interface

5.3 Sprint 3: Appointment Booking & Queue Generation

5.3.1 Sprint 3 Objective

The objective of Sprint 3 is to implement the **appointment booking module**, allowing the user to:

- Select a service within a healthcare establishment
- Submit an appointment request
- Receive a **queue number (turn_number)**
- Obtain an **estimated waiting time (estimated_time)**

This module ensures better organization of patient flow and reduces waiting time by introducing a structured queue management system.

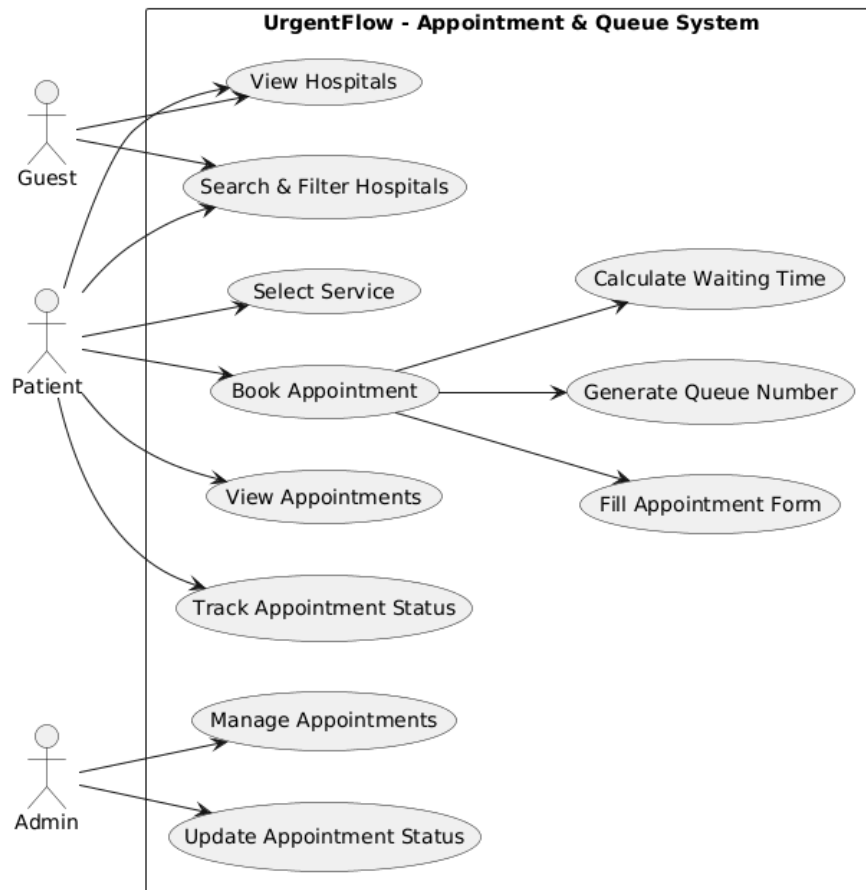
5.3.2 Sprint 3 Backlog

Id	Feature	User story	Priority	Complexity
1	Service Selection	As a user, I want to choose a service	High	Medium
2	Appointment Form	As a user, I want to enter my information and validate the booking	High	Medium
3	Appointment Creation	As a user, I want my appointment to be stored in the database	High	High
4	Queue Number	As a user, I want to receive a turn_number	High	High
5	Time Estimation	As a user, I want to receive an estimated waiting time	High	High
6	Appointment Status	As a user, I want to track my appointment status (waiting progress/done)	Medium	Medium

[Tab 5: Sprint 3 backlog table](#)

5.3.3 Sprint 3 Analysis

Use case diagram:



[Fig 16: Sprint 3 use case diagram](#)

Description of scenarios:

Scenario: Select Service

The user navigates to a healthcare establishment.

The system displays the list of available services (e.g., cardiology, emergency, general consultation).

The user selects the desired service.

Scenario: Fill Appointment Form

The user accesses the appointment form.

The user enters required information (date, time slot between 7am–12pm, notes).

The system validates the input data.

If valid, the user proceeds to submit the request.

Scenario: Book Appointment

The user submits the appointment request.

The system creates a new appointment in the database.

The appointment is linked to the selected service and the user.

Scenario: Generate Queue Number

Once the appointment is created, the system assigns a turn number (turn_number) based on the current queue for that service.

The turn number reflects the user's position in the waiting list.

Scenario: Estimate Waiting Time

The system calculates an estimated waiting time (estimated_time) based on:

- number of patients ahead in the queue
- average consultation duration

The estimated time is associated with the appointment.

Scenario: View Appointment Status

The user accesses the appointments page.

The system displays all appointments categorized by status:

- waiting
- in progress
- done
- cancelled

The user can track the evolution of each appointment.

5.3.4 Sprint 3 Design

Class Diagram:

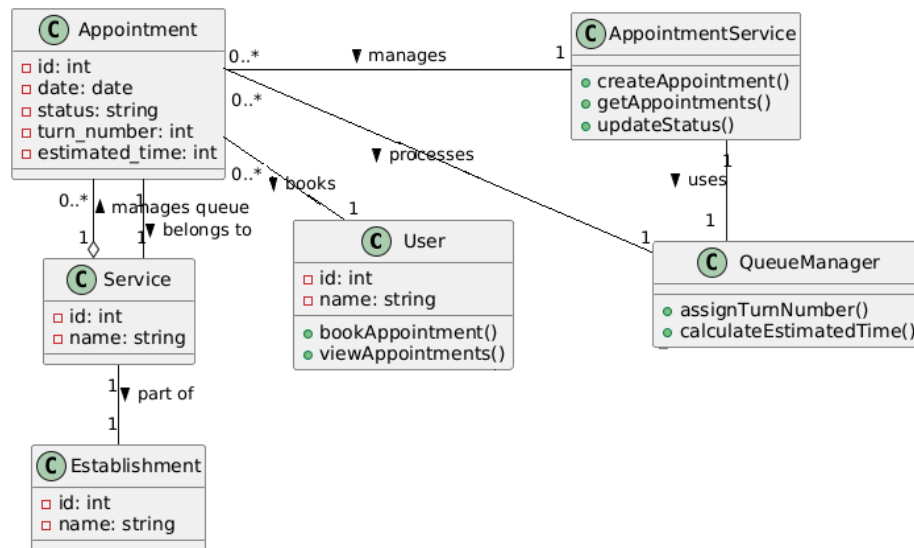


Fig 17: Sprint 3 class diagram

Class diagram description:

User

Represents a patient using the platform.

The user can create appointments, view their status, and interact with the booking system.

Establishment

Represents a healthcare facility where services are provided.

Each establishment contains multiple services available for booking.

Service

Represents a medical service within an establishment (e.g., emergency, consultation).

It is associated with a queue of appointments and defines the context for booking.

Appointment

Represents a booking made by a user.

It includes attributes such as:

- id
- date
- time slot
- status (waiting, in_progress, done, cancelled)

- turn_number
- estimated_time

It is the central entity of this module.

QueueManager

Handles the queue logic of the system.

It is responsible for:

- assigning turn numbers
- calculating estimated waiting time
- maintaining queue order

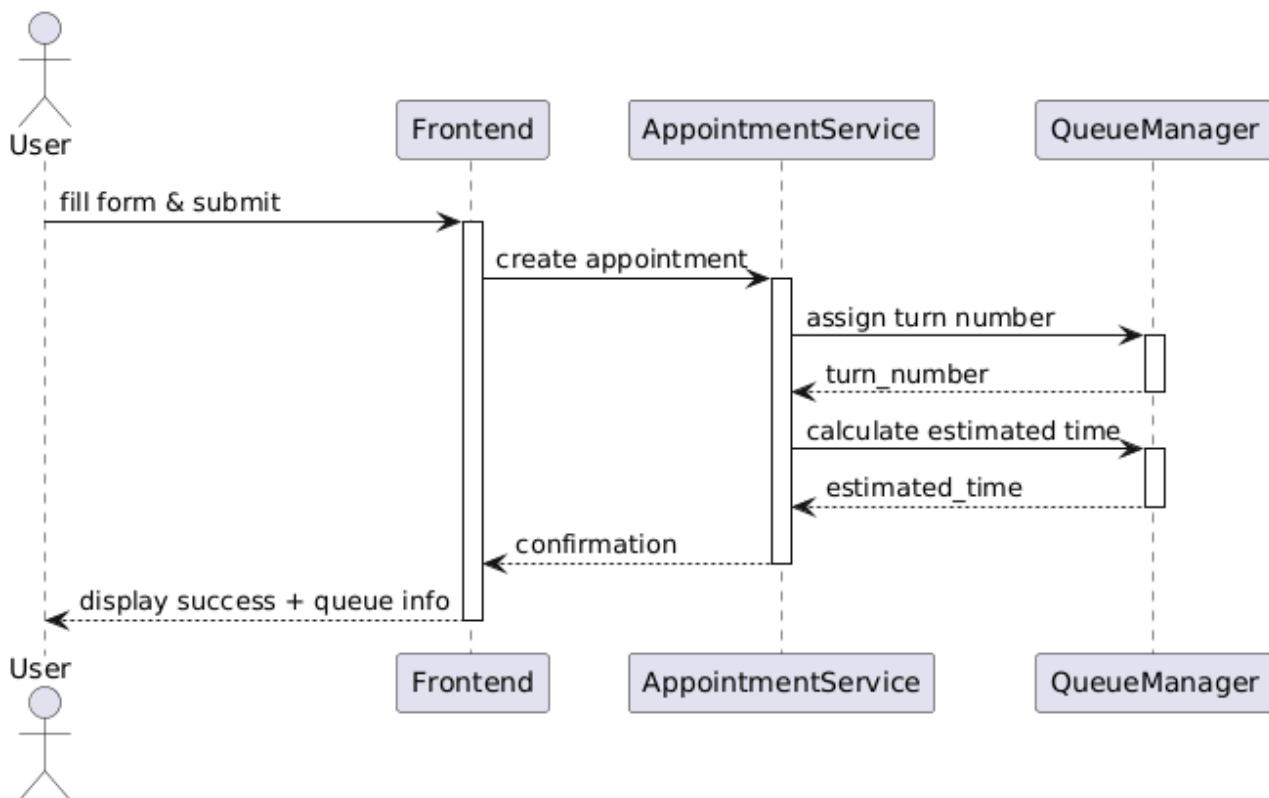
AppointmentService

Contains the business logic for appointment management.

It manages creation, retrieval, and status updates of appointments.

It interacts with the QueueManager to compute queue-related data.

Sprint 3 sequence diagrams:



[Fig 18: Appointment Booking Sequence Diagram](#)

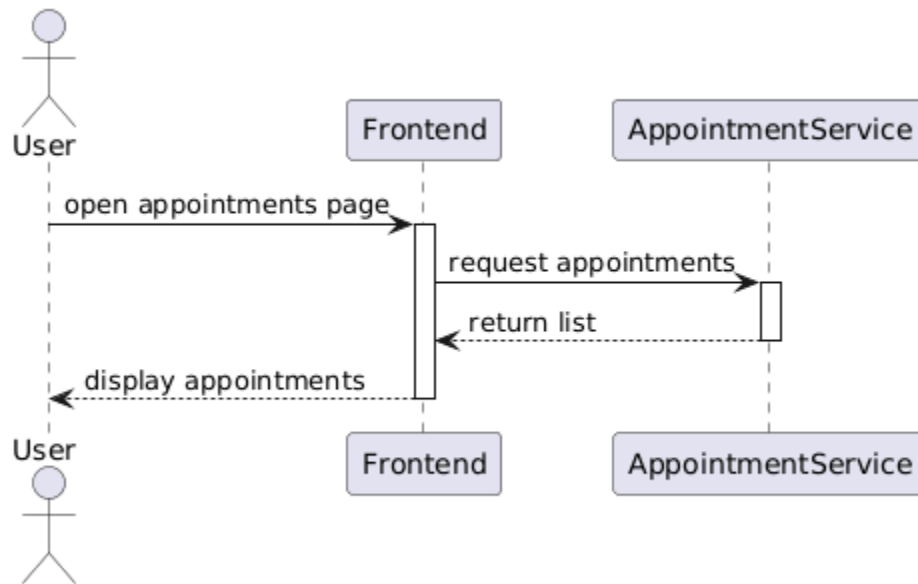


Fig 19: View Appointments Sequence Diagram

5.3.5 Completion of Sprint 3

Interfaces :



Book Appointment at La Rabta Hospital

Client Details

Name

Phone Number

Email

Select Service

Dermatology

Only today's bookings are allowed, and only between 07:00 and 12:00.

Confirm Place

Date

«
<
avril 2026
>
»

LUN	MAR	MER	JEU	VEN	SAM	DIM
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	1	2	3

Fig 20: Booking form interface

5.4 Sprint 4: Notifications & Emergency Management

5.4.1 Sprint 4 Objective

The objective of this sprint is to implement a real-time notification system and an emergency management module that allow:

- Notifying patients about their queue progress
- Sending alerts before their turn
- Allowing users to submit emergency requests
- Routing emergencies to the nearest healthcare facility
- Enabling emergency administrators to process urgent cases

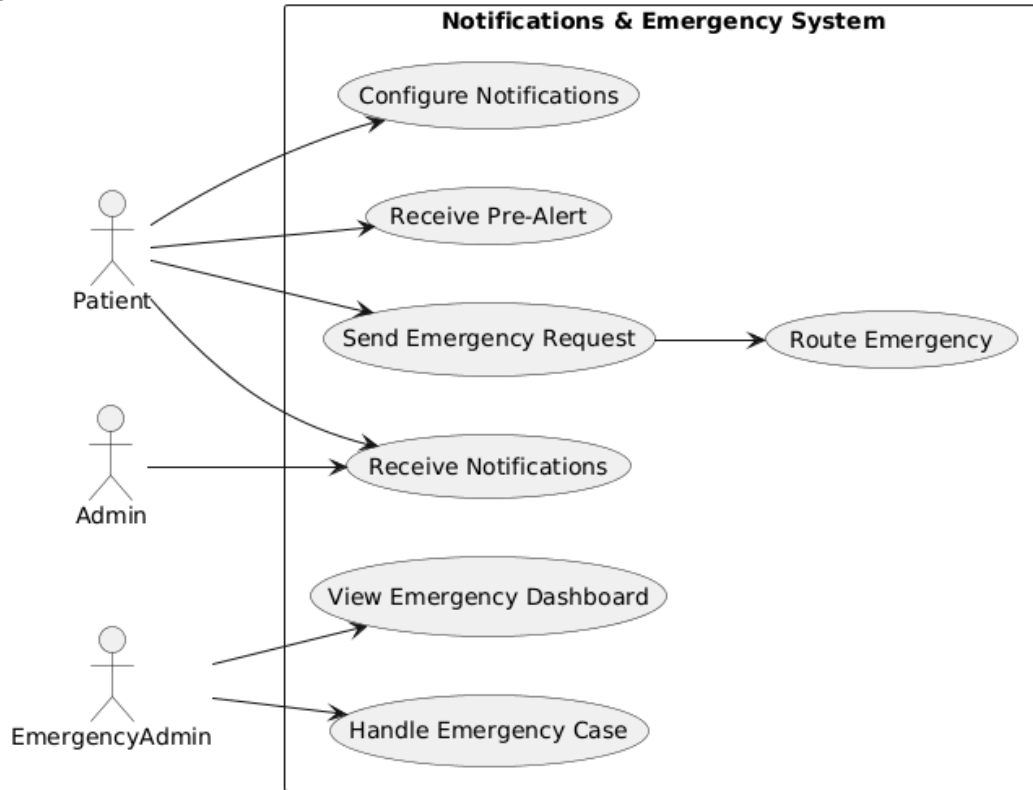
5.4.2 Sprint 4 Backlog

Id	Feature	User Story	Priority	Complexity
1	Real-time notifications	As a user, I want to receive updates about my appointment	High	High
2	Pre-alert notification	As a user, I want to be notified before my turn	High	Medium
3	Notification settings	As a user, I want to configure notification preferences	Medium	Medium
4	Emergency request	As a user, I want to send an emergency request	High	High
5	Emergency routing	As a system, I want to send the request to the nearest hospital	High	High
6	Emergency dashboard	As an admin, I want to manage emergency requests	High	High
7	Emergency handling	As an admin, I want to accept/reject emergency cases	High	Medium

[Tab 6: Sprint 4 backlog table](#)

5.4.3 Sprint 4 Analysis

Use case diagram:



[Fig 21: Sprint 4 use case diagram](#)

Description of scenarios:

Scenario: Receive Notifications

The system monitors the user's position in the queue.

When updates occur, the user receives a notification with relevant information.

Scenario: Pre-Alert Notification

Before the user's turn approaches, the system sends an alert based on configured timing (e.g., 10 minutes before).

Scenario: Configure Notifications

The user accesses settings.

The user selects notification preferences (method, timing).

The system stores the configuration.

Scenario: Send Emergency Request

The user presses the emergency button.

The system collects user location and request details.

The request is submitted for processing.

Scenario: Route Emergency

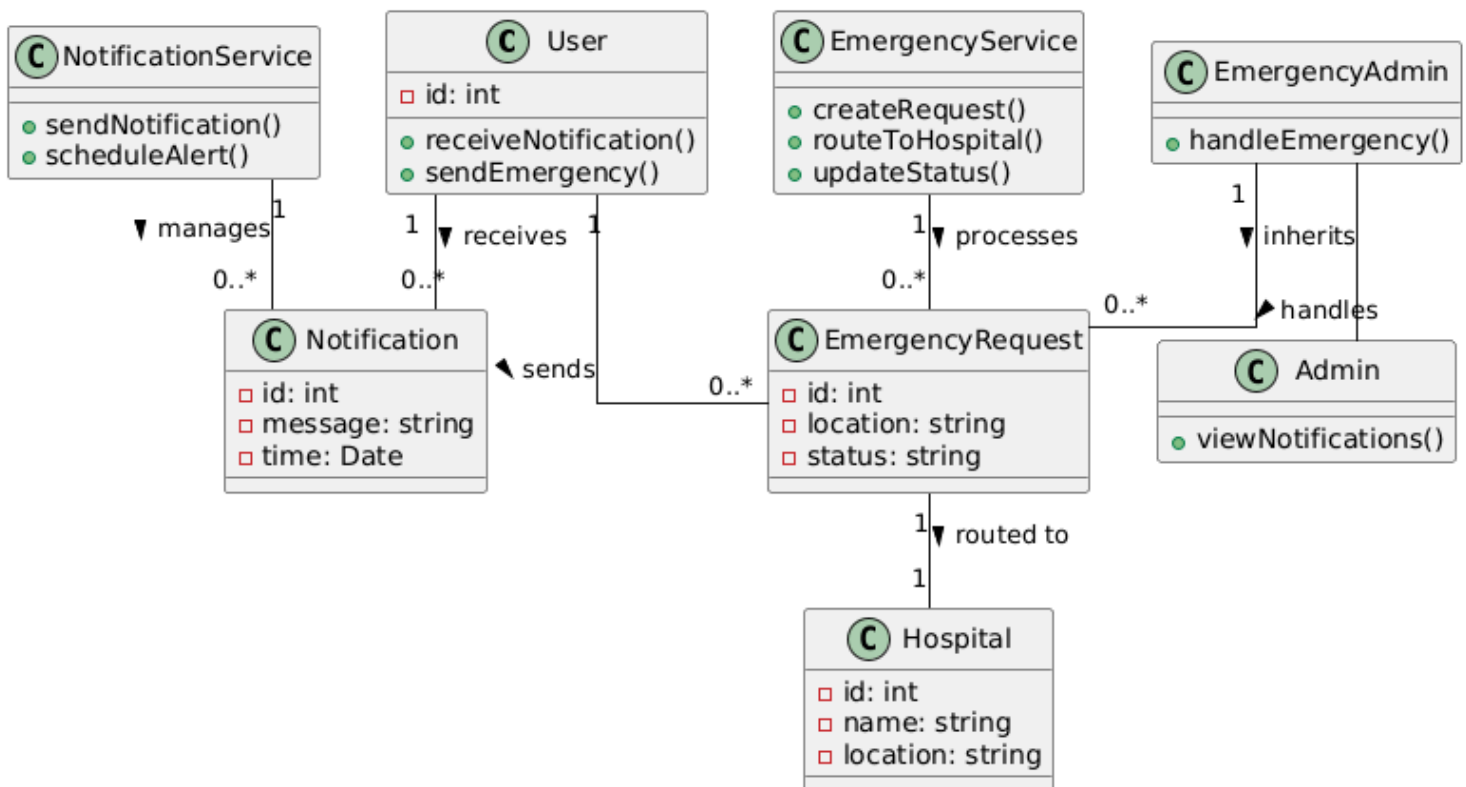
The system determines the nearest hospital using location data.
The request is forwarded to the appropriate emergency service.

Scenario: Handle Emergency Case

The emergency administrator accesses the dashboard.
The admin reviews incoming requests and decides to accept or reject them.
The system updates the request status.

5.4.4 Sprint 4 Design

Class Diagram:



[Fig 22: Sprint 4 class diagram](#)

Class Diagram Description

User

Represents the system user who can receive notifications and send emergency requests.

Admin

Represents a general administrator who can monitor system notifications.

EmergencyAdmin

A specialized administrator responsible for handling emergency cases.
It inherits from the Admin class.

Notification

Represents a system message sent to users, including alerts and updates.

NotificationService

Manages notification logic, including sending messages and scheduling alerts.

EmergencyRequest

Represents an emergency case submitted by a user, including location and status.

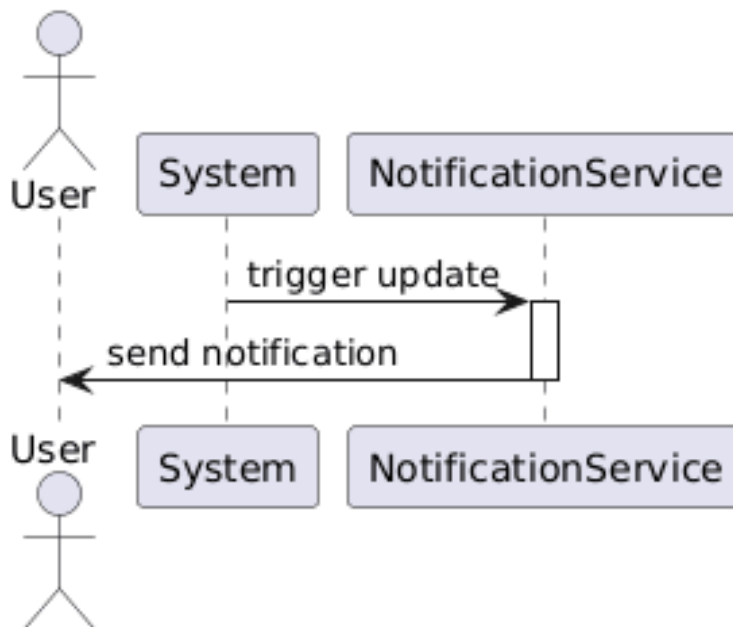
EmergencyService

Handles emergency request creation, routing to hospitals, and status updates.

Hospital

Represents a healthcare facility that receives emergency requests.

Sprint 4 sequence diagrams:



[Fig 23: Notification Flow Sequence Diagram](#)

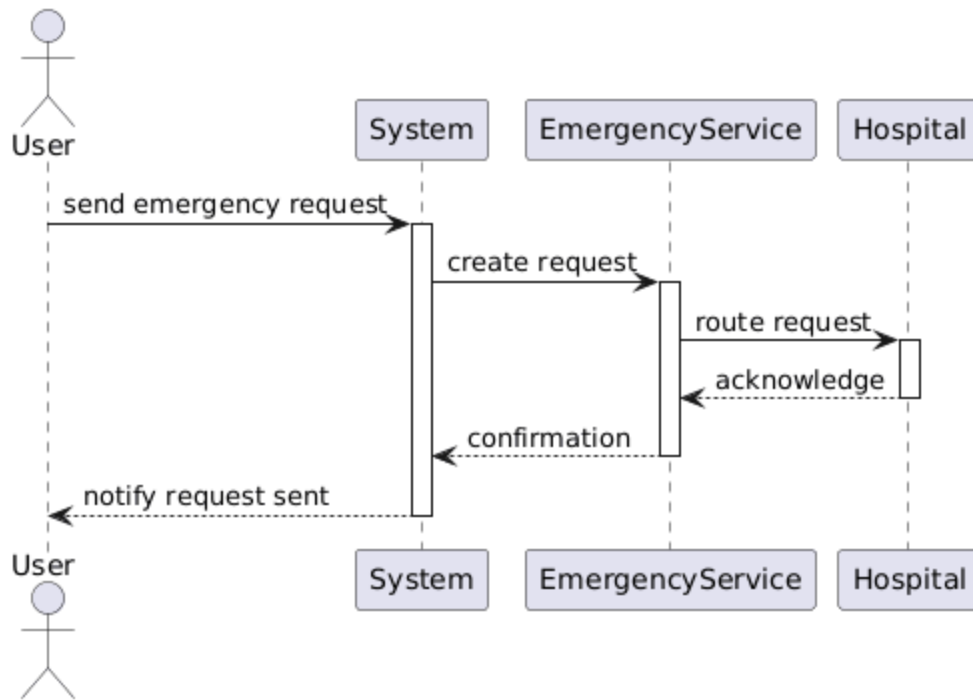


Fig 24: Emergency Request Flow Sequence Diagram

5.4.5 Completion of Sprint 4

Interfaces :

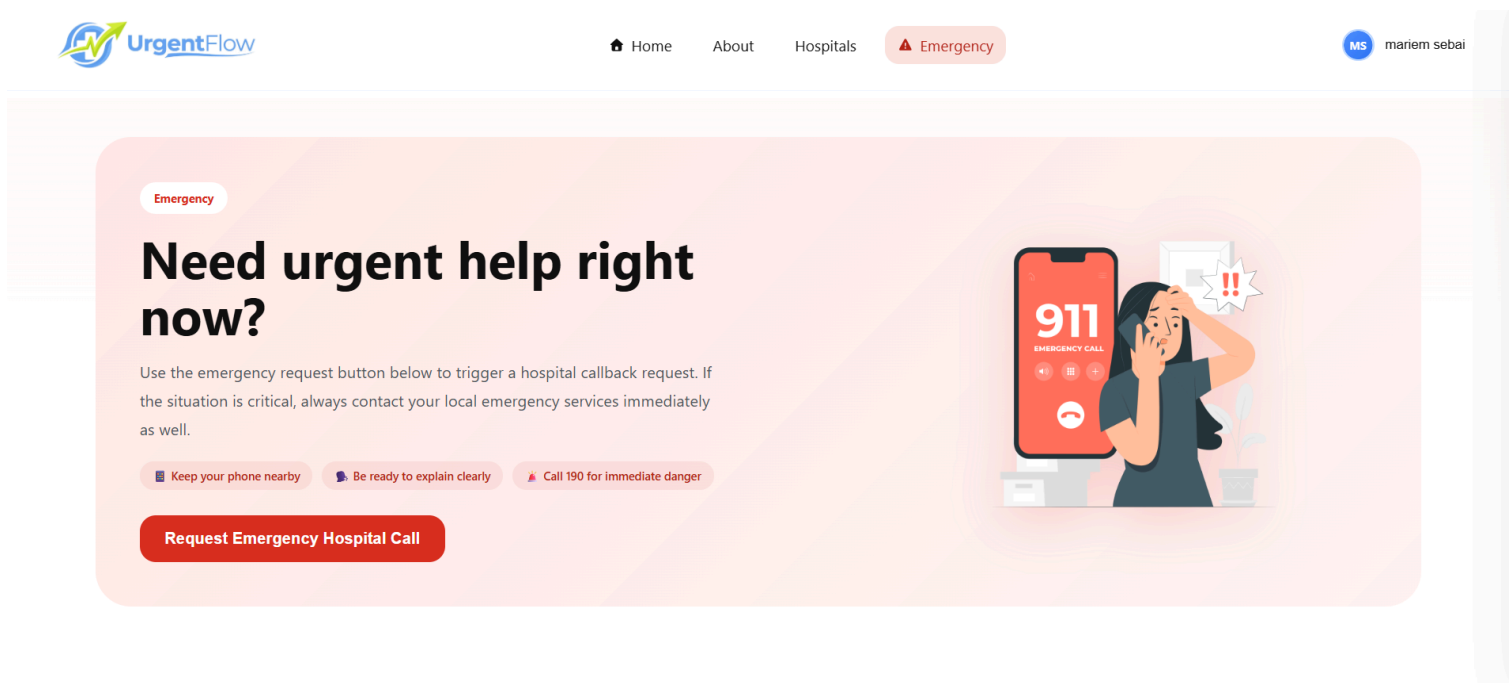


Fig 25: Emergency button interface

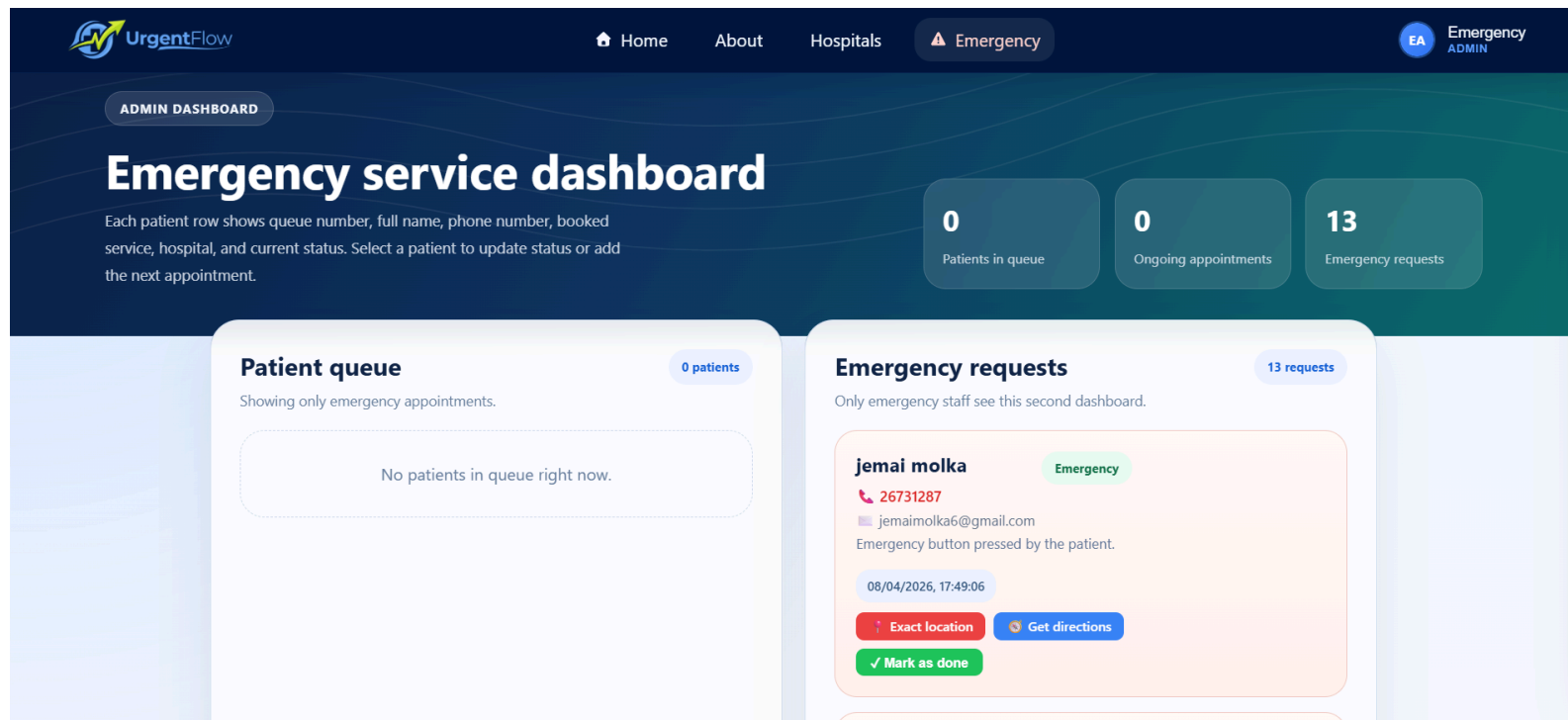


Fig 26: Emergency dashboard for administrators interface

5.5 Sprint 5: Administrative & Analytics Dashboard

5.5.1 Sprint 5 Objective

The objective of this sprint is to develop a comprehensive administrative and analytical dashboard that allows:

- Administrators to manage appointments and queues by service
- Emergency administrators to monitor and process urgent cases
- Super administrators to analyze user feedback and system performance
- The system to generate intelligent recommendations based on collected feedback

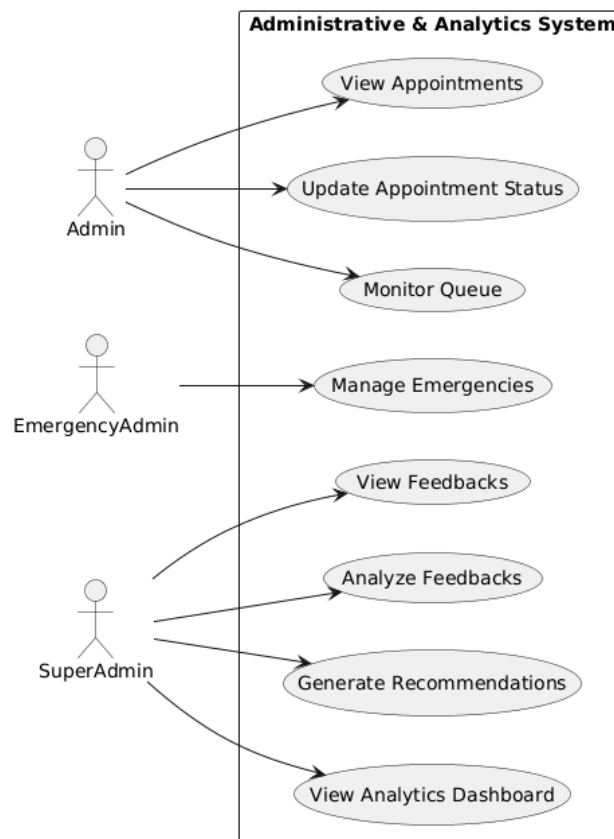
5.5.2 Sprint 5 backlog

Id	Feature	User Story	Priority	Complexity
1	Admin Dashboard	As an admin, I want to view appointments by service	High	Medium
2	Appointment Management	As an admin, I want to update appointment status	High	Medium
3	Queue Monitoring	As an admin, I want to track patient queues	High	High
4	Emergency Dashboard	As an emergency admin, I want to manage urgent cases	High	High
5	Multi-Service Panels	As an admin, I want a dashboard per service	Medium	High
6	Feedback Collection	As a system, I want to store user feedback	High	Medium
7	Analytics Dashboard	As a super admin, I want to view insights and trends	High	High
8	Smart Recommendations	As a system, I want to suggest improvements based on feedback	High	High

[Tab 7: Sprint 5 backlog table](#)

5.5.3 Sprint 5 Analysis

Use case diagram:



[Fig 27: Sprint 5 use case diagram](#)

Description of scenarios:

Scenario: View Appointments

The administrator accesses the dashboard.

The system displays appointments categorized by service and status.

Scenario: Update Appointment Status

The administrator selects an appointment.

The administrator updates its status (waiting, in progress, done, cancelled).

The system saves the modification.

Scenario: Monitor Queue

The administrator views the queue for a specific service.

The system displays patients ordered by their turn number.

Scenario: Manage Emergencies

The emergency administrator accesses the emergency dashboard.

The system displays incoming emergency requests.

The admin processes and updates their status.

Scenario: View Feedbacks

The super administrator accesses feedback data submitted by patients.

The system displays all feedback entries.

Scenario: Analyze Feedbacks

The system groups feedbacks based on frequency and patterns.

The super admin views categorized insights (common issues, ratings).

Scenario: Generate Recommendations

The system analyzes feedback trends.

It produces recommendations such as:

- service improvement suggestions
- common complaints resolution

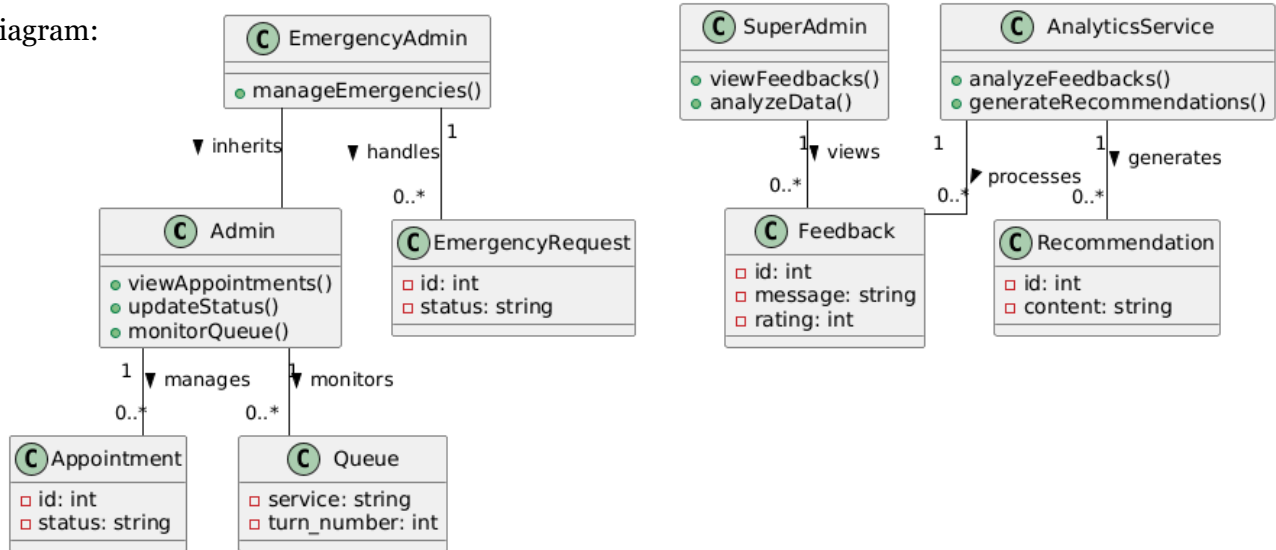
Scenario: View Analytics Dashboard

The super admin accesses a dashboard displaying:

- rating breakdown
- most frequent issues
- top signals of the week

5.5.4 Sprint 5 Design

Class Diagram:



[Fig 28: Sprint 5 class diagram](#)

Class Diagram Description

Admin

Manages appointments and monitors queues for a specific service.

EmergencyAdmin

A specialized admin responsible for handling emergency requests.
Inherits from Admin.

SuperAdmin

Responsible for system-wide supervision, including feedback analysis and performance insights.

Appointment

Represents a scheduled consultation with a status managed by admins.

Queue

Represents the ordered list of patients waiting for a service.

EmergencyRequest

Represents urgent cases requiring immediate handling.

Feedback

Represents user feedback submitted after appointments.

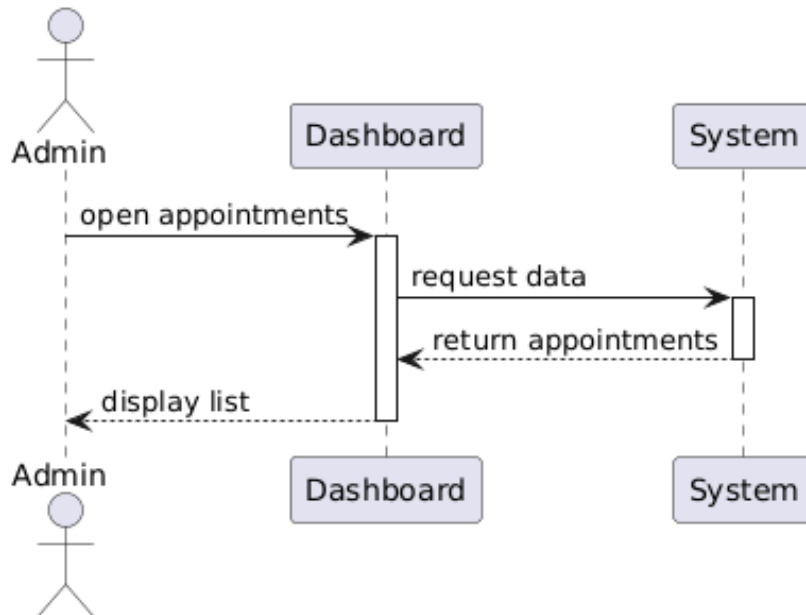
AnalyticsService

Processes feedback data and extracts insights.

Recommendation

Represents improvement suggestions generated from analyzed data.

Sprint 5 sequence diagrams:



[Fig 29: Appointment Management Sequence Diagram](#)

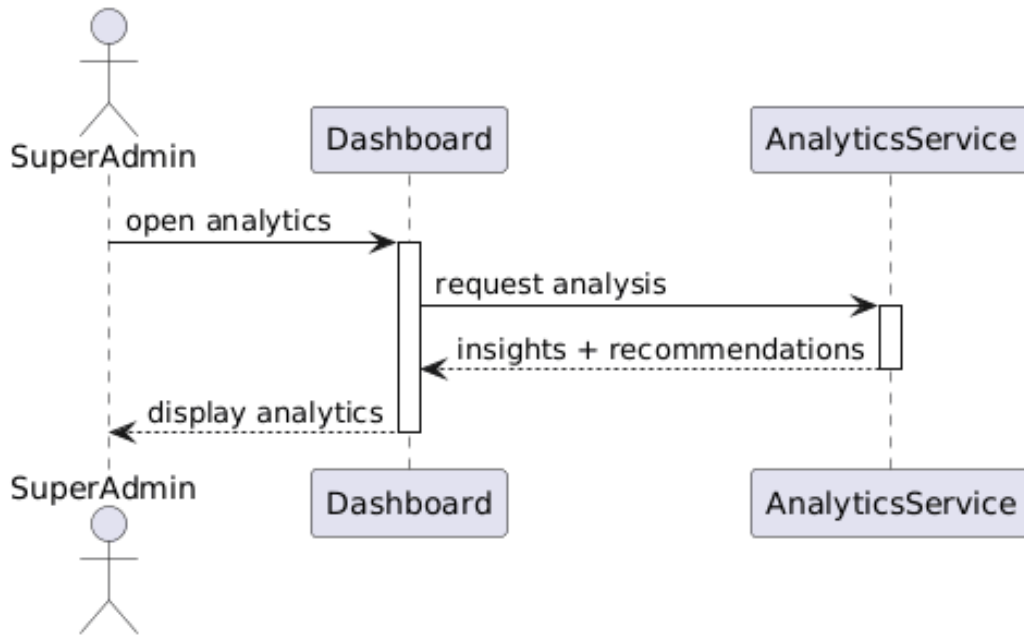


Fig 30: Feedback Analysis & Recommendations Sequence Diagram

5.5.5 Sprint 5 Completion

Interfaces :

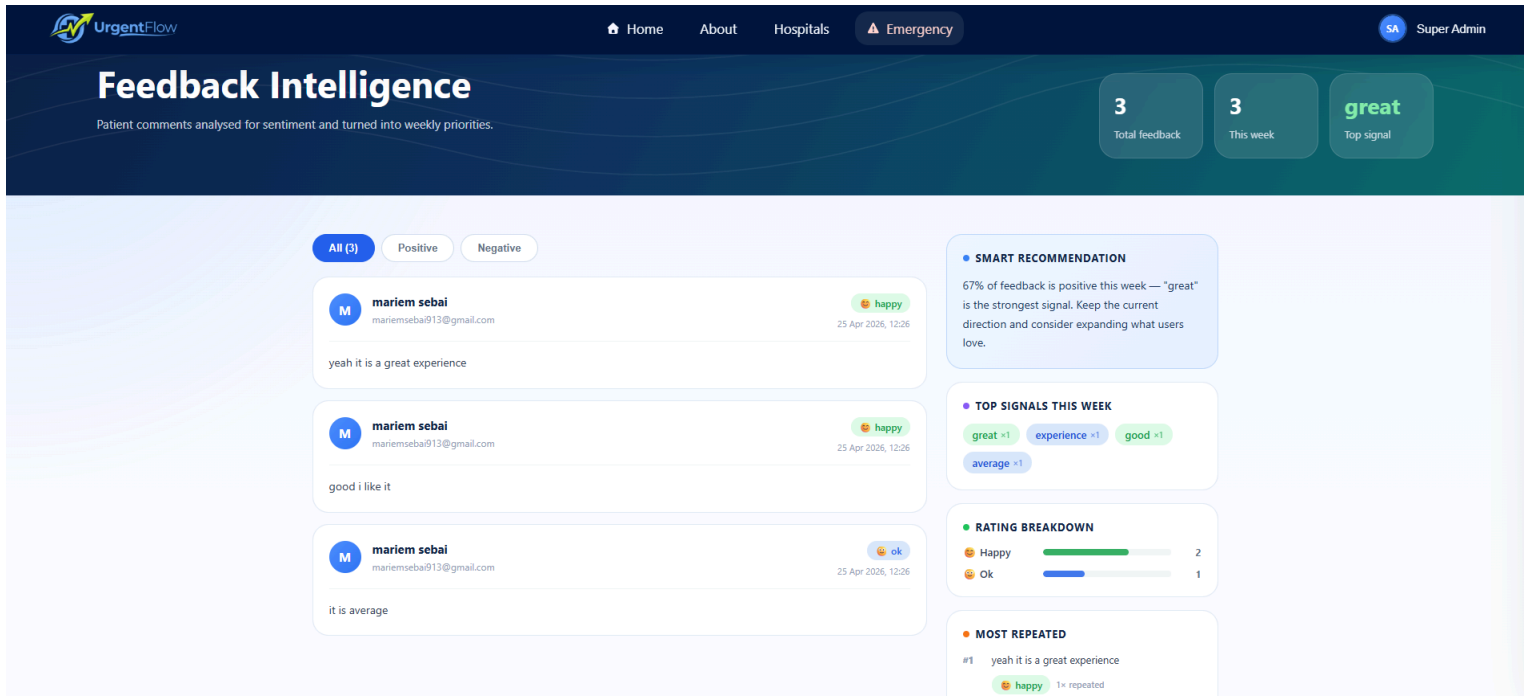


Fig 31: Feedback visualization interface

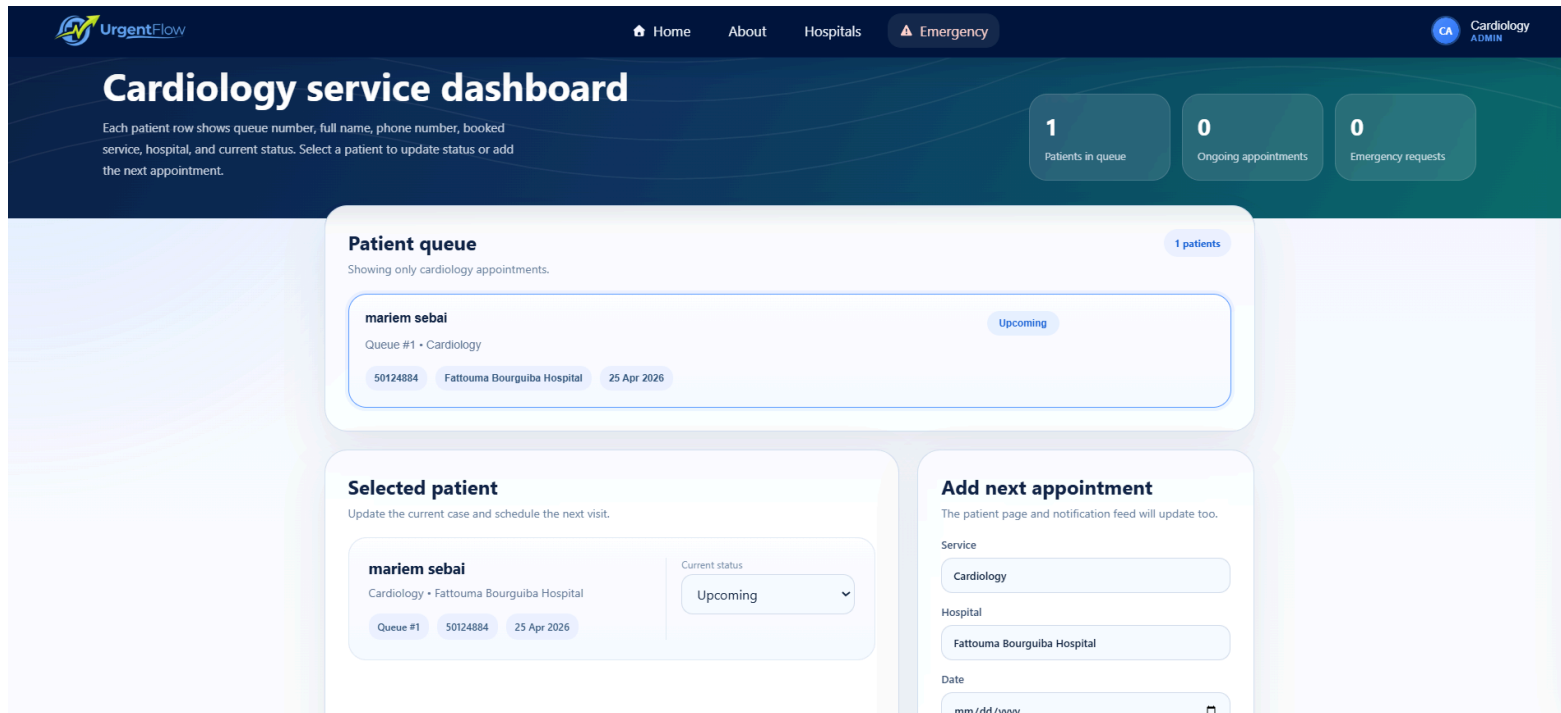


Fig 32: Admin dashboard interface

5.6 Conclusion :

This chapter presented the implementation of the **UrgentFlow** system through a series of structured sprints. Each sprint introduced key functionalities, including authentication, healthcare establishment management, appointment booking with queue generation, real-time notifications, emergency handling, and administrative and analytical dashboards.

The progressive development approach ensured the integration and validation of all system components, resulting in a coherent, efficient, and fully functional platform.

6. General Conclusion and Future Perspectives

This final-year project report presents the work carried out in the context of developing **UrgentFlow**, a smart web platform designed to improve the management of medical appointments and patient flow within healthcare institutions.

The main objective of this project was to design and implement a digital solution that addresses the limitations of traditional healthcare systems in Tunisia, particularly long waiting times, lack of visibility in queues, and inefficient handling of urgent cases. Through this project, we aimed to provide a system that allows patients to book appointments remotely, receive a queue number, estimate waiting time, and benefit from real-time updates.

To achieve this, we first analyzed the global context of the healthcare system, identifying its constraints, user needs, and organizational challenges. This allowed us to define a clear vision of the solution and to design a system adapted to real-world requirements.

We adopted an **Agile methodology based on SCRUM**, which enabled iterative development and continuous improvement throughout the project lifecycle. The system was divided into multiple sprints, each focusing on a specific module: authentication, healthcare establishment integration, appointment booking and queue generation, notifications and emergency management, and administrative and analytical dashboards.

The implementation phase detailed the development of each module, including UML modeling, system architecture, and user interface design. Special attention was given to the development of:

- A **queue management system** with turn numbers and time estimation
- A **real-time notification system** to improve patient awareness
- An **emergency routing module** for urgent case handling
- A **multi-level administrative dashboard**, including a **Super Admin analytics system** capable of generating smart recommendations based on patient feedback

This project allowed us to apply our theoretical knowledge to a real-world problem while developing strong technical and organizational skills in software engineering, system design, and teamwork.

The proposed solution significantly improves:

- Patient experience (reduced waiting time, better visibility)
- Hospital organization (efficient flow management)
- Decision-making (data-driven insights and recommendations)

We believe that **UrgentFlow** represents a valuable contribution to the modernization of healthcare services and has the potential to positively impact both patients and healthcare providers.

Future Perspectives

❖ *Short-term Improvements*

- **Enhanced Notification System**

Introduce more advanced notifications (SMS, push notifications, real-time updates) to improve communication with patients.

- **Feedback Enhancement Module**

Improve the feedback system by allowing structured feedback (ratings per service, categorized issues).

- **Performance Optimization**

Optimize API performance and database queries to handle a larger number of concurrent users.

❖ *Long-term Improvements*

- **AI-based Medical Triage Agent (Your idea — VERY STRONG)**

Integrate an intelligent agent capable of analyzing patient symptoms before booking.

This agent would:

- Evaluate the urgency level of the case
- Prioritize critical patients in the queue
- Recommend immediate emergency routing if necessary

- **Predictive Queue Management (Very valuable addition)**

Use machine learning to:

- Predict waiting times more accurately
- Optimize appointment scheduling
- Reduce congestion in peak hours

- **Internationalization**

Extend the platform to support multiple languages and adapt it to healthcare systems in other countries.

- **Mobile Application Development**

Develop a dedicated mobile application to enhance accessibility and real-time interaction.

- **Integration with Electronic Health Records (EHR)**

Connect the platform with hospital medical systems to:

- Access patient history
- Improve diagnosis and appointment prioritization

Fin du rapport